Use Cases Points

for

CampusSphere – Campus Opportunity Hub, Release 1.0

Version 1.2 approved

Rajwardhan Singh - 24CSB1A52
Ramitha Ramesh - 24CSB1A53
Rayi Charan Harsha - 24CSB1A54

Prepared by CampusSphere Project Team

12/Oct/26

USE CASE POINTS (UCP) MANUAL CALCULATION

Unadjusted Actor Weight (UAW)

Actors	Complexity	Weight
Student	Complex	3
Faculty	Complex	3
Recruiter	Complex	3
Admin	Complex	3
Email/ SMS Notification Gateway	Simple	1

UAW Calculation

Simple actors: 2 x 1 = 2
Average actors: 0 x 2 = 0
Complex actors: 4 x 3 = 12
UAW (Unadjusted Actor Weight) = 14

Unadjusted Use Case Weight (UUCW)

Use Case ID	Use Case Name	Transactions	Complexity	Weight
1	Complete Onboarding Survey	8	Complex	15
2	Receive Personalized Opportunity Recommendations	8	Complex	15
3	Apply for an Opportunity & Track Application Status	9	Complex	15
4	Post an Opportunity for Approval	7	Average	10
5	Bookmark Opportunity & Receive Deadline Reminders	5	Average	10
6	Approve Opportunity Postings & Moderate Platform	6	Average	10
7	Edit Skill Profile & Resume	6	Average	10
8	Search and Filter Opportunities Manually	5	Average	10
9	Comment on an Opportunity	6	Average	10
10	Withdraw a Submitted Application	6	Average	10
11	Manage Categories and Tags	6	Average	10
12	Manage Notification Preferences	6	Average	10
13	View Achievements and Redeem Rewards	6	Average	10

14	Verify Organization Account	6	Average	10
15	View Platform Analytics Dashboard	4	Average	10
16	Reset Forgotten Password	6	Average	10
17	Generate Applicant Report for a Posting	5	Average	10

UUCW Calculation

Simple use cases: 0 x 5 = 0

Average use cases: 14 x 10 = 140

Complex use cases: 3 x 15 = 45

UUCW (Unadjusted Use Case Weight) = 185

Technical Complexity Factor (TCF)

Code Factor	Weight	Value	W*V
T1 Distributed system	2.0	3	6.0
T2 Response time / performance objectives	1.0	4	4.0
T3 End-user efficiency	1.0	4	4.0
T4 Internal processing complexity	1.0	3	3.0
T5 Code reusability	1.0	3	3.0
T6 Easy to install	0.5	4	2.0
T7 Easy to use	0.5	4	2.0
T8 Portability to other platforms	2.0	4	8.0
T9 System maintenance	1.0	3	3.0
T10 Concurrent / parallel processing	1.0	3	3.0
T11 Security features	1.0	4	4.0
T12 Access for third parties	1.0	3	3.0
T13 End-user training	1.0	2	2.0

TCF Calculation

TF (sum) = 47.0

TCF = 0.6 + (TF/100) = 0.6 + (47.0/100) = 1.0700

Environmental Complexity Factor (ECF)

Code Factor	Weight	Value	W*V
E1 Familiarity with development process used	1.5	3	4.5
E2 Application experience	0.5	2	1.0
E3 Object-oriented experience of team	1.0	3	3.0
E4 Lead analyst capability	0.5	3	1.5
E5 Motivation of the team	1.0	4	4.0
E6 Stability of requirements	2.0	3	6.0
E7 Part-time staff	-1.0	4	-4.0
E8 Difficult programming language	-1.0	1	-1.0

ECF Calculation

EF (sum) = 15.0

ECF = 1.4 + (-0.03 x EF) = 1.4 + (-0.03 x 15.0) = 0.9500

Final UCP Calculation

UCP = (UUCW + UAW) x TCF x ECF

UCP = (185 + 14) x 1.0700 x 0.9500

UCP = 202.28

Effort Estimate

Estimated Effort = UCP x hours/UCP = 202.28 x 20

Estimated Effort = **4045.7 person-hours** (~506 person-days, ~25.3 person-months)

Code:

```
from dataclasses import dataclass, field
from typing import List

# ============================================================================
# 1. Standard UCP Calculation Engine
# ============================================================================

ACTOR_WEIGHTS = {
    "simple": 1,    # External system, interacts via a well-defined API
    "average": 2,   # External system, interacts via a standard protocol
(TCP/IP, HTTP, DB)
    "complex": 3,   # Human actor, interacts via a GUI
}

USE_CASE_WEIGHTS = {
    "simple": 5,    # 1-3 transactions
    "average": 10,  # 4-7 transactions
    "complex": 15,  # 8+ transactions
}

TCF_FACTORS = [
    ("T1",  "Distributed system",                2.0),
    ("T2",  "Response time / performance objectives", 1.0),
    ("T3",  "End-user efficiency",               1.0),
    ("T4",  "Internal processing complexity",    1.0),
    ("T5",  "Code reusability",                  1.0),
    ("T6",  "Easy to install",                   0.5),
    ("T7",  "Easy to use",                       0.5),
    ("T8",  "Portability to other platforms",    2.0),
    ("T9",  "System maintenance",                1.0),
    ("T10", "Concurrent / parallel processing",  1.0),
    ("T11", "Security features",                 1.0),
    ("T12", "Access for third parties",          1.0),
    ("T13", "End-user training",                 1.0),
]

ECF_FACTORS = [
    ("E1", "Familiarity with development process used", 1.5),
    ("E2", "Application experience",                    0.5),
```

```
    ("E3", "Object-oriented experience of team",          1.0),
    ("E4", "Lead analyst capability",                     0.5),
    ("E5", "Motivation of the team",                      1.0),
    ("E6", "Stability of requirements",                   2.0),
    ("E7", "Part-time staff",                            -1.0),
    ("E8", "Difficult programming language",             -1.0),
]


@dataclass
class UseCase:
    name: str
    transactions: int  # Number of main-flow steps / transactions

    def classification(self):
        if self.transactions <= 3:
            return "simple"
        elif self.transactions <= 7:
            return "average"
        else:
            return "complex"

    def weight(self):
        return USE_CASE_WEIGHTS[self.classification()]


@dataclass
class Actor:
    name: str
    classification: str  # "simple" | "average" | "complex"

    def weight(self):
        return ACTOR_WEIGHTS[self.classification]


@dataclass
class UCPInputs:
    actors: List[Actor] = field(default_factory=list)
    use_cases: List[UseCase] = field(default_factory=list)
    tcf_values: dict = field(default_factory=dict)   # {"T1": 0..5, ...}
```

```
    ecf_values: dict = field(default_factory=dict)    # {"E1": 0..5, ...}
    hours_per_ucp: float = 20.0


def compute_uaw(actors: List[Actor]):
    breakdown = {"simple": 0, "average": 0, "complex": 0}
    for a in actors:
        breakdown[a.classification] += 1
    uaw = sum(breakdown[c] * ACTOR_WEIGHTS[c] for c in breakdown)
    return uaw, breakdown


def compute_uucw(use_cases: List[UseCase]):
    breakdown = {"simple": 0, "average": 0, "complex": 0}
    for uc in use_cases:
        breakdown[uc.classification()] += 1
    uucw = sum(breakdown[c] * USE_CASE_WEIGHTS[c] for c in breakdown)
    return uucw, breakdown


def compute_tcf(tcf_values: dict):
    rows = []
    tf_total = 0.0
    for code, desc, weight in TCF_FACTORS:
        assigned = tcf_values.get(code, 0)
        contribution = weight * assigned
        tf_total += contribution
        rows.append((code, desc, weight, assigned, contribution))
    tcf = 0.6 + (tf_total / 100.0)
    return tcf, tf_total, rows


def compute_ecf(ecf_values: dict):
    rows = []
    ef_total = 0.0
    for code, desc, weight in ECF_FACTORS:
        assigned = ecf_values.get(code, 0)
        contribution = weight * assigned
        ef_total += contribution
        rows.append((code, desc, weight, assigned, contribution))
```

```
    ecf = 1.4 + (-0.03 * ef_total)
    return ecf, ef_total, rows


def calculate_ucp(inputs: UCPInputs):
    uaw, actor_breakdown = compute_uaw(inputs.actors)
    uucw, uc_breakdown = compute_uucw(inputs.use_cases)
    tcf, tf_total, tcf_rows = compute_tcf(inputs.tcf_values)
    ecf, ef_total, ecf_rows = compute_ecf(inputs.ecf_values)

    ucp = (uucw + uaw) * tcf * ecf
    effort_hours = ucp * inputs.hours_per_ucp

    return {
        "uaw": uaw, "actor_breakdown": actor_breakdown,
        "uucw": uucw, "uc_breakdown": uc_breakdown,
        "tcf": tcf, "tf_total": tf_total, "tcf_rows": tcf_rows,
        "ecf": ecf, "ef_total": ef_total, "ecf_rows": ecf_rows,
        "ucp": ucp, "effort_hours": effort_hours,
        "hours_per_ucp": inputs.hours_per_ucp,
    }


def print_report(inputs: UCPInputs, result: dict):
    sep = "=" * 72
    print(sep)
    print("USE CASE POINTS (UCP) CALCULATION REPORT")
    print(sep)

    print("\n--- Actors ---")
    for a in inputs.actors:
        print(f"  {a.name:<40} {a.classification:<10}
weight={a.weight()}")
    b = result["actor_breakdown"]
    print(f"\n  Simple actors:  {b['simple']} x 1 = {b['simple']*1}")
    print(f"  Average actors: {b['average']} x 2 = {b['average']*2}")
    print(f"  Complex actors: {b['complex']} x 3 = {b['complex']*3}")
    print(f"  UAW (Unadjusted Actor Weight) = {result['uaw']}")

    print("\n--- Use Cases ---")
```

```
    for uc in inputs.use_cases:
        print(f"  {uc.name:<48} {uc.transactions:>2} txns  "
              f"-> {uc.classification():<8} weight={uc.weight()}")
    c = result["uc_breakdown"]
    print(f"\n  Simple use cases:  {c['simple']} x 5  = {c['simple']*5}")
    print(f"  Average use cases: {c['average']} x 10 = {c['average']*10}")
    print(f"  Complex use cases: {c['complex']} x 15 = {c['complex']*15}")
    print(f"  UUCW (Unadjusted Use Case Weight) = {result['uucw']}")

    print("\n--- Technical Complexity Factor (TCF) ---")
    print(f"
{'Code':<5}{'Factor':<40}{'Weight':>7}{'Value':>7}{'W*V':>8}")
    for code, desc, weight, assigned, contrib in result["tcf_rows"]:
        print(f"
{code:<5}{desc:<40}{weight:>7}{assigned:>7}{contrib:>8}")
    print(f"  TF (sum) = {result['tf_total']}")
    print(f"  TCF = 0.6 + (TF/100) = 0.6 + ({result['tf_total']}/100) =
{result['tcf']:.4f}")

    print("\n--- Environmental Complexity Factor (ECF) ---")
    print(f"
{'Code':<5}{'Factor':<40}{'Weight':>7}{'Value':>7}{'W*V':>8}")
    for code, desc, weight, assigned, contrib in result["ecf_rows"]:
        print(f"
{code:<5}{desc:<40}{weight:>7}{assigned:>7}{contrib:>8}")
    print(f"  EF (sum) = {result['ef_total']}")
    print(f"  ECF = 1.4 + (-0.03 x EF) = 1.4 + (-0.03 x
{result['ef_total']}) = {result['ecf']:.4f}")

    print("\n--- Final UCP Calculation ---")
    print(f"  UCP = (UUCW + UAW) x TCF x ECF")
    print(f"  UCP = ({result['uucw']} + {result['uaw']}) x
{result['tcf']:.4f} x {result['ecf']:.4f}")
    print(f"  UCP = {result['ucp']:.2f}")

    print("\n--- Effort Estimate ---")
    print(f"  Estimated Effort = UCP x hours/UCP = {result['ucp']:.2f} x
{result['hours_per_ucp']}")
    print(f"  Estimated Effort = {result['effort_hours']:.1f} person-hours
"
```

```
          f"(~{result['effort_hours']/8:.0f} person-days, "
          f"~{result['effort_hours']/(8*20):.1f} person-months)")
    print(sep)


# ============================================================================
# 2. CampusSphere Data & Execution
# ============================================================================

actors = [
    Actor("Student",                                "complex"),
    Actor("Faculty",                                "complex"),
    Actor("Recruiter",                              "complex"),
    Actor("Admin",                                  "complex"),
    Actor("Email/SMS Notification Gateway (API)", "simple"),
    Actor("AI Recommendation API (optional)",     "simple"),
]

use_cases = [
    UseCase("1. Complete Onboarding Survey",                        8),
    UseCase("2. Receive Personalized Opportunity Recommendations",  8),
    UseCase("3. Apply for an Opportunity & Track Application Status", 9),
    UseCase("4. Post an Opportunity for Approval",                  7),
    UseCase("5. Bookmark Opportunity & Receive Deadline Reminders", 5),
    UseCase("6. Approve Opportunity Postings & Moderate Platform",  6),
    UseCase("7. Edit Skill Profile & Resume",                       6),
    UseCase("8. Search and Filter Opportunities Manually",          5),
    UseCase("9. Comment on an Opportunity",                         6),
    UseCase("10. Withdraw a Submitted Application",                 6),
    UseCase("11. Manage Categories and Tags",                       6),
    UseCase("12. Manage Notification Preferences",                  6),
    UseCase("13. View Achievements and Redeem Rewards",             6),
    UseCase("14. Verify Organization Account",                      6),
    UseCase("15. View Platform Analytics Dashboard",                4),
    UseCase("16. Reset Forgotten Password",                         6),
    UseCase("17. Generate Applicant Report for a Posting",          5),
]

tcf_values = {
    "T1": 3,   # Distributed system: web+admin clients, API, DB, rec.
```

```
engine
    "T2": 4,   # Response time objectives: 2s search/filter target
    "T3": 4,   # End-user efficiency: mobile-first, simple UX
    "T4": 3,   # Internal processing complexity: recommendation scoring
    "T5": 3,   # Code reusability: REST API reused by web client + admin
console
    "T6": 4,   # Easy to install: Dockerized
    "T7": 4,   # Easy to use: simple, mobile-friendly UI
    "T8": 4,   # Portability: browser-based + Docker
    "T9": 3,   # System maintenance: containerized and documented
    "T10": 3,  # Concurrent/parallel processing: peak loads near deadlines
    "T11": 4,  # Security features: JWT, RBAC, HTTPS, hashed passwords
    "T12": 3,  # Access for third parties: Recruiters/external orgs
    "T13": 2,  # End-user training: minimal training needed
}

ecf_values = {
    "E1": 3,   # Familiarity with development process (Agile/SDLC)
    "E2": 2,   # Application experience
    "E3": 3,   # OO experience of team
    "E4": 3,   # Lead analyst capability
    "E5": 4,   # Motivation of team
    "E6": 3,   # Stability of requirements
    "E7": 4,   # Part-time staff: team works around full course load
    "E8": 1,   # Difficult programming language: Python/JS
}
HOURS_PER_UCP = 20

def main():
    inputs = UCPInputs(
        actors=actors,
        use_cases=use_cases,
        tcf_values=tcf_values,
        ecf_values=ecf_values,
        hours_per_ucp=HOURS_PER_UCP,
    )
    result = calculate_ucp(inputs)
    print_report(inputs, result)
if __name__ == "__main__":
    main()
```

Output:

```
================================================================================
USE CASE POINTS (UCP) CALCULATION REPORT
================================================================================

--- Actors ---
  Student                                  complex    weight=3
  Faculty                                  complex    weight=3
  Recruiter                                complex    weight=3
  Admin                                    complex    weight=3
  Email/SMS Notification Gateway (API)     simple     weight=1
  AI Recommendation API (optional)         simple     weight=1

  Simple actors:  2 x 1 = 2
  Average actors: 0 x 2 = 0
  Complex actors: 4 x 3 = 12
  UAW (Unadjusted Actor Weight) = 14

--- Use Cases ---
  1. Complete Onboarding Survey                    8 txns  -> complex
weight=15
  2. Receive Personalized Opportunity Recommendations  8 txns  -> complex
weight=15
  3. Apply for an Opportunity & Track Application Status  9 txns  ->
complex  weight=15
  4. Post an Opportunity for Approval              7 txns  -> average
weight=10
  5. Bookmark Opportunity & Receive Deadline Reminders  5 txns  -> average
weight=10
  6. Approve Opportunity Postings & Moderate Platform  6 txns  -> average
weight=10
  7. Edit Skill Profile & Resume                   6 txns  -> average
weight=10
  8. Search and Filter Opportunities Manually      5 txns  -> average
weight=10
  9. Comment on an Opportunity                     6 txns  -> average
weight=10
  10. Withdraw a Submitted Application             6 txns  -> average
weight=10
```

```
  11. Manage Categories and Tags                    6 txns  -> average
weight=10
  12. Manage Notification Preferences               6 txns  -> average
weight=10
  13. View Achievements and Redeem Rewards          6 txns  -> average
weight=10
  14. Verify Organization Account                   6 txns  -> average
weight=10
  15. View Platform Analytics Dashboard             4 txns  -> average
weight=10
  16. Reset Forgotten Password                      6 txns  -> average
weight=10
  17. Generate Applicant Report for a Posting       5 txns  -> average
weight=10

  Simple use cases:  0 x 5  = 0
  Average use cases: 14 x 10 = 140
  Complex use cases: 3 x 15 = 45
  UUCW (Unadjusted Use Case Weight) = 185

--- Technical Complexity Factor (TCF) ---
  Code Factor                                 Weight  Value     W*V
  T1   Distributed system                        2.0      3     6.0
  T2   Response time / performance objectives    1.0      4     4.0
  T3   End-user efficiency                       1.0      4     4.0
  T4   Internal processing complexity            1.0      3     3.0
  T5   Code reusability                          1.0      3     3.0
  T6   Easy to install                           0.5      4     2.0
  T7   Easy to use                               0.5      4     2.0
  T8   Portability to other platforms            2.0      4     8.0
  T9   System maintenance                        1.0      3     3.0
  T10  Concurrent / parallel processing          1.0      3     3.0
  T11  Security features                         1.0      4     4.0
  T12  Access for third parties                  1.0      3     3.0
  T13  End-user training                         1.0      2     2.0
  TF (sum) = 47.0
  TCF = 0.6 + (TF/100) = 0.6 + (47.0/100) = 1.0700

--- Environmental Complexity Factor (ECF) ---
  Code Factor                                 Weight  Value     W*V
```

```
  E1   Familiarity with development process used    1.5      3     4.5
  E2   Application experience                       0.5      2     1.0
  E3   Object-oriented experience of team           1.0      3     3.0
  E4   Lead analyst capability                      0.5      3     1.5
  E5   Motivation of the team                       1.0      4     4.0
  E6   Stability of requirements                    2.0      3     6.0
  E7   Part-time staff                             -1.0      4    -4.0
  E8   Difficult programming language              -1.0      1    -1.0
  EF (sum) = 15.0
  ECF = 1.4 + (-0.03 x EF) = 1.4 + (-0.03 x 15.0) = 0.9500

--- Final UCP Calculation ---
  UCP = (UUCW + UAW) x TCF x ECF
  UCP = (185 + 14) x 1.0700 x 0.9500
  UCP = 202.28

--- Effort Estimate ---
  Estimated Effort = UCP x hours/UCP = 202.28 x 20
  Estimated Effort = 4045.7 person-hours (~506 person-days, ~25.3
person-months)
==========================================================================
```